FITBUDDY

AI Fitness Plan Generator using Gemini Models

S. A. COLLEGE OF ARTS & SCIENCE

B.Sc. Computer Science

PROJECT REPORT

Submitted by	Mithra Venkatesh
Register Number	222510966
Programme / Department	B.Sc. Computer Science
College	S. A. College of Arts & Science
Academic Year	2026 – 2027

A software project report presenting the design and development of an AI-assisted fitness planning application.

ABSTRACT

FitBuddy is an AI-assisted fitness planning application designed to generate structured and adaptable fitness suggestions from user-provided information. The system uses Gemini models as the generative AI component to interpret user inputs and produce an organized plan through a simple application interface. The project demonstrates how generative AI can be integrated with conventional software components such as user interfaces, input validation, prompt construction, response handling, and data storage.

The application is designed around a responsible approach: the generated content is treated as general informational guidance rather than a substitute for professional medical or fitness advice. The system can include safety notices, input validation, and guardrails to avoid inappropriate or unsafe recommendations. The project focuses primarily on software design, AI integration, usability, and response generation.

Keywords: Artificial Intelligence, Gemini Models, Generative AI, Fitness Planner, Prompt Engineering, Python, Web Application, Personalization.

TABLE OF CONTENTS

1	Introduction
2	Problem Statement
3	Objectives
4	Existing System
5	Proposed System
6	System Requirements
7	System Design and Architecture
8	Modules
9	AI Integration and Prompt Design
10	Implementation Methodology
11	Testing
12	Advantages and Limitations
13	Future Enhancements
14	Conclusion
15	References

1. INTRODUCTION

Fitness planning often requires users to organize goals, preferences, available time, activity levels, and other constraints into a consistent routine. Traditional applications generally provide fixed workout libraries or manually selected schedules. Generative AI introduces another approach: an application can interpret structured inputs and generate a human-readable plan dynamically.

FitBuddy is proposed as a software application that combines a user interface with a Gemini model to generate a personalized-looking fitness plan from user inputs. The system can collect non-sensitive planning information such as fitness goal, available days, preferred activity types, approximate session duration, and experience level. The application then constructs a controlled prompt and sends it to the AI service.

The project is primarily an academic demonstration of AI application development. It emphasizes prompt engineering, modular design, usability, responsible AI use, and clear presentation of generated output.

1.1 Scope

- Collect user inputs through a simple and understandable interface.
- Validate required fields before sending a request to the AI model.
- Generate a structured fitness-planning response using a Gemini model.
- Display the generated plan in a readable format.
- Include safety notes and encourage professional guidance where appropriate.

2. PROBLEM STATEMENT

Many beginner-oriented fitness applications either rely on static plans or require users to manually select and combine activities. This can make planning less convenient, especially when a user wants a plan that reflects several constraints at once. There is therefore an opportunity to demonstrate an AI-based system that accepts natural-language or structured inputs and produces an organized response.

The problem addressed by FitBuddy is: **How can a software application use a generative AI model to transform user fitness-planning inputs into a clear, structured, and responsibly presented fitness plan?**

The project does not attempt to diagnose medical conditions or replace qualified professionals. Instead, it demonstrates an AI-powered planning workflow with appropriate limitations and safety messaging.

3. OBJECTIVES

- To design an easy-to-use AI-assisted fitness planning application.
- To integrate a Gemini model into a software application.
- To apply prompt engineering for consistent and structured outputs.
- To validate user input and handle incomplete or invalid requests.
- To present AI-generated results in a clear and readable format.
- To include responsible-use safeguards and limitation notices.
- To demonstrate a practical use case of generative AI in an academic project.

4. EXISTING SYSTEM

Existing fitness applications commonly use predefined workout databases, fixed schedules, calculators, or manually selected exercises. Such systems can be useful, but they may require users to browse several screens and assemble a plan themselves.

Common characteristics

- Static or rule-based workout suggestions.
- Large exercise libraries requiring manual selection.
- Limited natural-language interaction.
- Personalization based mainly on predefined categories.

5. PROPOSED SYSTEM

FitBuddy introduces a generative-AI layer. Instead of selecting every item manually, the user supplies planning information and the application creates a structured response using a Gemini model. The generated output can be organized into sections such as weekly schedule, session focus, general activity suggestions, recovery reminders, and safety notes.

The proposed system separates application logic from AI generation. This allows the interface and validation layer to remain predictable while the model handles language generation. The design also makes it easier to add additional validation, logging, or storage later.

6. SYSTEM REQUIREMENTS

6.1 Hardware Requirements

- Computer or laptop with a modern processor.
- At least 4 GB RAM; 8 GB or more is recommended for development.
- Stable internet connection for cloud-based AI model access.

6.2 Software Requirements

- Operating system: Windows, Linux, or macOS.
- Python 3.x for backend development.
- A web framework such as Flask or FastAPI, if a web-based implementation is selected.
- HTML, CSS, and JavaScript for a web interface, where applicable.
- Gemini API / Google AI SDK configured according to the current official documentation.
- A code editor such as Visual Studio Code.

6.3 Functional Requirements

- The system shall accept user planning inputs.
- The system shall validate mandatory fields.
- The system shall construct a controlled prompt.
- The system shall request a response from the Gemini model.
- The system shall display the generated result.
- The system shall handle API or network errors gracefully.

7. SYSTEM DESIGN AND ARCHITECTURE

The architecture follows a simple client–application–AI-service flow. The user interacts with the interface, the application validates and prepares the input, and the Gemini model generates the requested content. The response is then processed and displayed.

7.1 Data Flow

- User enters planning information.
- Application validates and normalizes the input.
- Prompt builder creates a structured instruction for the model.
- Gemini model generates the response.
- Application receives and processes the response.
- Final plan and safety notice are shown to the user.

8. MODULES

8.1 User Input Module

Collects information such as goal category, experience level, available days, preferred activity type, and approximate session duration. The interface should avoid requesting unnecessary sensitive information.

8.2 Validation Module

Checks whether required fields are present and whether values are within acceptable application-defined ranges. Invalid input is returned to the user with a clear message instead of being sent directly to the model.

8.3 Prompt Engineering Module

Converts structured data into a consistent prompt. The prompt can define the expected output sections, tone, constraints, and safety requirements.

8.4 Gemini Integration Module

Handles authentication, request construction, model invocation, response retrieval, and API error handling. API keys or credentials must be stored securely and must not be hard-coded into public source code.

8.5 Result Display Module

Formats the model response into headings, lists, tables, or cards so that users can understand the generated plan without reading a large block of text.

8.6 Safety and Error Handling Module

Adds general safety messaging, handles unavailable services, manages timeouts, and prevents the application from displaying malformed or empty results as if they were valid plans.

9. AI INTEGRATION AND PROMPT DESIGN

The main AI component is the Gemini model. The application does not simply pass raw user text to the model; instead, it can create a structured prompt containing the purpose, user-provided planning information, output format, and responsible-use instructions.

Example prompt structure

Role: You are an assistant generating general fitness-planning information.
Inputs: goal, experience level, available days, session duration, preferences.
Task: Produce a clear weekly plan in a structured format.
Output: headings, day-wise sections, general reminders, and a safety note.
Constraint: Do not diagnose medical conditions or present the output as professional medical advice.

Prompt design principles

- Be explicit about the requested output structure.
- Separate user data from system instructions.
- Avoid unnecessary collection of sensitive personal information.
- Tell the model what it should not claim or diagnose.
- Use consistent headings so the response can be displayed predictably.
- Handle model uncertainty and service errors in the application layer.

10. IMPLEMENTATION METHODOLOGY

The project can be developed using an iterative software-development process. The first stage is requirements analysis, followed by interface design, AI integration, validation, testing, and refinement.

10.1 Development Steps

- 1. Define the user inputs and expected output format.
- 2. Create the user interface.
- 3. Implement input validation.
- 4. Create the prompt builder.
- 5. Connect the application to the Gemini service using the current official SDK/API method.
- 6. Process the generated response.
- 7. Add error handling and responsible-use messaging.
- 8. Test with normal, incomplete, invalid, and service-failure cases.
- 9. Refine the interface and output formatting.

10.2 Security Considerations

- Keep API credentials in environment variables or a secure secret-management mechanism.
- Do not expose private keys in client-side code or public repositories.
- Collect only information required for the application.
- Avoid storing personal data unless storage is genuinely required and appropriately protected.

11. TESTING

Testing verifies that the application accepts valid inputs, rejects inappropriate inputs, communicates correctly with the AI service, and presents useful responses.

Sample Test Cases

TC01: All required fields valid → AI request is generated and response is displayed.
TC02: Required field missing → Validation message is shown; request is not sent.
TC03: Invalid duration/value → Input is rejected or corrected according to defined rules.
TC04: AI service unavailable → Friendly error message is shown.
TC05: Empty AI response → Application does not display an empty result as successful.
TC06: Unexpected model formatting → Response is handled safely and displayed in a readable fallback format.

12. ADVANTAGES AND LIMITATIONS

12.1 Advantages

- Natural-language generation makes the application interactive.
- Plans can be generated dynamically instead of relying only on fixed templates.
- The architecture is modular and can be extended with additional features.
- Prompt-based output can be formatted into clear sections.
- The project demonstrates a practical application of generative AI.

12.2 Limitations

- AI-generated content can contain mistakes or unsuitable suggestions.
- The application depends on network connectivity and AI-service availability.
- Model output may vary between requests.
- The quality of generated output depends on input quality and prompt design.
- A general-purpose AI model is not a substitute for professional medical or fitness assessment.

13. FUTURE ENHANCEMENTS

- Add user accounts and secure plan history.
- Provide progress tracking using user-entered activity data.
- Add multilingual support for a wider audience.
- Introduce selectable output formats such as weekly cards or downloadable reports.
- Add a rule-based validation layer before and after AI generation.
- Provide configurable reminders and calendar integration.
- Add accessibility improvements such as larger text and screen-reader-friendly layouts.
- Evaluate model outputs systematically using predefined quality and safety test cases.

14. CONCLUSION

FitBuddy demonstrates how generative AI can be integrated into a practical software application to transform user inputs into structured fitness-planning information. By combining a simple interface, validation, prompt engineering, Gemini model integration, response formatting, and responsible-use safeguards, the project provides a clear example of an AI-enabled application architecture.

The key learning outcomes include understanding API-based AI integration, designing effective prompts, handling generated responses, building modular application components, and testing AI-powered software. The same architecture can be adapted to other domains where users need structured information generated from a controlled set of inputs.

15. REFERENCES

- Google AI for Developers – Gemini documentation and API/SDK guidance.
- Python Software Foundation – Python documentation.
- Flask / FastAPI official documentation, depending on the implementation selected.
- General software engineering references on requirements analysis, modular design, testing, and secure API usage.